FoFa – Technical Design Document

Version: 1.0
Date: 2026-04-15
Audience: Engineering, Product, Design, QA
Status: Baseline

Table of Contents

---

1. Overview

FoFa (Foster Families) is a community platform for foster families. It provides a shared space where verified members can post announcements, exchange direct messages, manage family profiles, discover other community members, and coordinate playdates.

The platform is built as a monorepo containing an Express REST API, a React single-page application, a React Native mobile app, and a Cypress end-to-end test suite.

Core Feature Set

Feature	Description
Auth	Email-verified registration, JWT-based sessions, password reset
Announcements feed	Create/edit/delete posts with optional image or video attachments
Comments and reactions	Threaded comments and five reaction types per announcement
Direct messages	Private one-to-one messaging with unread counts
Family profiles	Per-member cards with name, age, and photo
Community search	Search all verified users by name
Playdates	Availability slot management and request/accept/decline flow
User profiles	Edit display name, city, state, and thumbnail

---

2. Goals and Non-Goals

Goals

- Provide a safe, verified community space limited to registered foster families.
- Keep the infrastructure simple enough to run on a single server without managed cloud services.
- Enable rapid iteration: local development requires only Node.js and a Gmail App Password.
- Cover all critical user flows with automated tests (unit, integration, and E2E).

Non-Goals

- Real-time features (WebSockets, push notifications) are not in scope for this version.
- Group messaging (more than two participants) is not implemented.
- The mobile app (`mobile/`) shares the same API but is treated as a separate delivery vehicle; it is not fully feature-parity with the web app in this baseline.
- Multi-tenancy or organization isolation — every verified user is in the same community.

---

3. System Architecture

3.1 Repository Layout

```
fofa/
├── backend/        # Express 4 + TypeScript REST API  (port 4000)
├── frontend/       # React 18 + TypeScript SPA        (port 5173)
├── mobile/         # React Native / Expo app          (Expo Go)
└── cypress/        # End-to-end test suite
```

3.2 Runtime Topology

```
Browser (React SPA)
        │  HTTP + Axios
        ▼
  Vite dev proxy  ──► Express API (port 4000)
                              │
                      SQLite file (fofa.db)
                              │
                      ./uploads/   (local disk)
                              │
                      Gmail SMTP   (Nodemailer)
```

In production the Vite proxy is replaced by a reverse-proxy rule (e.g., Nginx) that forwards `/api` and `/uploads` to the Express process. The SPA's static build is served from the same Nginx instance.

3.3 Technology Choices

Concern	Choice	Rationale
API runtime	Node.js 18 + Express 4	Minimal footprint, large ecosystem
API language	TypeScript (strict)	Type safety across the full stack
Database	SQLite via `better-sqlite3`	Zero-dependency persistence; synchronous API simplifies controllers
Frontend framework	React 18	Mature, team familiarity
Frontend bundler	Vite 5	Fast HMR; built-in test runner integration

Concern	Choice	Rationale
Server state	TanStack Query v5	Declarative caching, pagination, and invalidation
Client state	Zustand	Tiny footprint; persists auth token to localStorage
Styling	Tailwind CSS 3 + CSS variables	Utility-first; dark/light mode via `class` strategy
Forms	React Hook Form	Uncontrolled inputs, minimal re-renders
HTTP client	Axios	Interceptors for JWT injection and 401 handling
Mobile	React Native / Expo	Code-share with web types and API layer

---

4. Data Model

All tables are created on startup via `runMigrations()` in `backend/src/utils/migrate.ts`. The database runs in WAL mode with foreign keys enforced.

4.1 Entity Relationship Summary

```
users
 ├── email_tokens          (1:many, on-delete cascade)
 ├── password_reset_tokens (1:many, on-delete cascade)
 ├── family_members        (1:many, on-delete cascade)
 ├── announcements         (1:many, on-delete cascade)
 │    ├── comments         (1:many, on-delete cascade)
 │    └── reactions        (1:many, on-delete cascade; unique per user per announcement)
 ├── messages              (sender + receiver, both FK to users)
 └── availability_slots    (1:many, on-delete cascade)
      └── playdate_requests (1:many, on-delete cascade)
```

4.2 Table Definitions

`users`

Column	Type	Notes
id	TEXT PK	UUID v4

Column	Type	Notes
email	TEXT UNIQUE NOT NULL	Lowercase, normalized
password	TEXT NOT NULL	bcrypt, 12 rounds
name	TEXT NOT NULL	Display name
city	TEXT NOT NULL	
state	TEXT NOT NULL	
thumbnail	TEXT	Relative path under `/uploads/thumbnails/`
verified	INTEGER	0 = unverified, 1 = verified; login blocked until 1
created_at / updated_at	TEXT	ISO 8601 from `datetime('now')`

`email_tokens`

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	Cascade delete
token	TEXT UNIQUE NOT NULL	UUID v4 sent in verification link
expires_at	TEXT NOT NULL	24 hours after creation
used	INTEGER	Single-use; set to 1 on consumption

`password_reset_tokens`

Same shape as `email_tokens` with a 1-hour expiry.

`family_members`

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
name	TEXT NOT NULL	
age	INTEGER NOT NULL	0–120 validated at API
thumbnail	TEXT	`/uploads/thumbnails/`

`announcements`

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
content	TEXT NOT NULL	
media_url	TEXT	`/uploads/media/<uuid>.<ext>`
media_type	TEXT	`'image'` or `'video'` ; CHECK constraint

`comments`

Column	Type	Notes
id	TEXT PK	UUID v4
announcement_id	TEXT FK → announcements	
user_id	TEXT FK → users	
content	TEXT NOT NULL	

`reactions`

Column	Type	Notes
id	TEXT PK	UUID v4

Column	Type	Notes
announcement_id	TEXT FK → announcements	
user_id	TEXT FK → users	
type	TEXT	CHECK IN ('like','love','hug','celebrate','support')
—	—	UNIQUE (announcement_id, user_id) — one reaction per user

`messages`

Column	Type	Notes
id	TEXT PK	UUID v4
sender_id	TEXT FK → users	
receiver_id	TEXT FK → users	
content	TEXT NOT NULL	
read	INTEGER	0 = unread, 1 = read; marked when recipient fetches thread

`availability_slots`

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
date	TEXT NOT NULL	YYYY-MM-DD
start_time / end_time	TEXT NOT NULL	HH:MM
status	TEXT	CHECK IN ('free','busy') DEFAULT 'free'
note	TEXT	Optional free-text

`playdate_requests`

Column	Type	Notes
id	TEXT PK	UUID v4
requester_id	TEXT FK → users	
owner_id	TEXT FK → users	Slot owner
slot_id	TEXT FK → availability_slots	
message	TEXT	Optional context from requester
status	TEXT	CHECK IN ('pending','accepted','declined') DEFAULT 'pending'

4.3 Indexes

```
idx_announcements_user        ON announcements(user_id)
idx_announcements_created     ON announcements(created_at DESC)
idx_comments_announcement     ON comments(announcement_id)
idx_reactions_announcement    ON reactions(announcement_id)
idx_messages_sender           ON messages(sender_id)
idx_messages_receiver         ON messages(receiver_id)
idx_family_user               ON family_members(user_id)
idx_slots_user_date           ON availability_slots(user_id, date)
idx_requests_requester        ON playdate_requests(requester_id)
idx_requests_owner            ON playdate_requests(owner_id)
```

---

5. API Design

Base path: `/api`

All protected routes require `Authorization: Bearer <jwt>` header.

5.1 Auth (`/api/auth`)

Method	Path	Auth	Description
POST	`/register`	No	Create account; sends verification email

Method	Path	Auth	Description
GET	`/verify-email?token=`	No	Consume email token, set verified=1
POST	`/login`	No	Returns JWT + user object
POST	`/forgot-password`	No	Sends reset link (always 200 to prevent enumeration)
POST	`/reset-password`	No	Consumes reset token, updates password
POST	`/change-password`	Yes	Validates current password before update

Registration input validation (express-validator): email format, password min 8 chars, name/city/state non-empty.

5.2 Users (`/api/users`)

Method	Path	Auth	Description
GET	`/me`	Yes	Returns current user's profile
PUT	`/me`	Yes	Update profile fields + optional thumbnail upload
GET	`/search?q=`	Yes	Search users by name (partial match)
GET	`/:id`	Yes	Get any user's public profile

5.3 Family (`/api/family`)

Method	Path	Auth	Description
GET	`/`	Yes	List all family members for current user
POST	`/`	Yes	Add a family member (multipart with optional thumbnail)
PUT	`/:id`	Yes	Update a member
DELETE	`/:id`	Yes	Remove a member

5.4 Announcements (`/api/announcements`)

Method	Path	Auth	Description
GET	`/`	Yes	Paginated list (20/page, newest first) with reactions + comment count
POST	`/`	Yes	Create with optional media attachment (up to 100 MB)
PUT	`/:id`	Yes	Update content (own announcements only)
DELETE	`/:id`	Yes	Delete (own announcements only)
GET	`/:id/comments`	Yes	List all comments for an announcement
POST	`/:id/comments`	Yes	Add a comment
DELETE	`/:id/comments/:commentId`	Yes	Delete own comment
POST	`/:id/reactions`	Yes	Toggle reaction (add/change/remove)

Reaction toggle behavior: if the user has no reaction, it is added. If the same type is sent again, it is removed. If a different type is sent, the existing reaction is updated.

5.5 Messages (`/api/messages`)

Method	Path	Auth	Description
GET	`/`	Yes	List conversations (latest message per partner)
GET	`/:partnerId`	Yes	Get paginated thread; marks messages as read
POST	`/`	Yes	Send a message
GET	`/unread/count`	Yes	Count unread messages for navbar badge

Note on route ordering: `/unread/count` must be registered before `/:partnerId` in the router to prevent Express from matching `"unread"` as a partner ID. The current `routes/index.ts` registers conversations and messages before the unread route — this is a known ordering issue that works in practice because the unread endpoint is declared after `/:partnerId` but is matched correctly due to Express's exact string matching on `"unread"`.

5.6 Playdates (`/api/playdates`)

Method	Path	Auth	Description
GET	`/availability/:userId`	Yes	Get slots; own view shows all statuses, others see only 'free'
POST	`/availability`	Yes	Add a slot
PUT	`/availability/:id`	Yes	Update own slot
DELETE	`/availability/:id`	Yes	Delete own slot
GET	`/requests`	Yes	All requests where current user is requester or owner
POST	`/requests`	Yes	Request a free slot (duplicate pending blocked)
PUT	`/requests/:id/respond`	Yes	Accept or decline (owner only)

5.7 Response Shapes

Pagination envelope (announcements):

```
{
  "data": [...],
  "pagination": { "page": 1, "limit": 20, "total": 42, "pages": 3 }
}
```

Error responses follow `{ "error": "<message>" }` with appropriate HTTP status codes (400, 401, 403, 404, 409, 413, 500).

---

6. Frontend Architecture

6.1 Application Shell

`App.tsx` wraps the entire tree in `QueryClientProvider` (TanStack Query) and `BrowserRouter`. Two layout wrappers handle route guarding:

- `GuestLayout` — redirects to `/dashboard` if a token is present.
- `ProtectedLayout` — redirects to `/login` if no token is present; renders the `Navbar`.

`QueryClient` is configured with `retry: 1` and `staleTime: 30,000 ms`.

6.2 Route Map

Path	Component	Protected
`/login`	LoginPage	No
`/register`	RegisterPage	No
`/forgot-password`	ForgotPasswordPage	No
`/reset-password`	ResetPasswordPage	No
`/verify-email`	VerifyEmailPage	No (public)
`/dashboard`	DashboardPage	Yes
`/family`	FamilyPage	Yes
`/messages`	MessagesPage	Yes
`/community`	CommunityPage	Yes
`/profile`	ProfilePage	Yes
`/members/:id`	MemberProfilePage	Yes
`/playdates`	PlaydatesPage	Yes
`*` / `/`	Redirect to `/dashboard`	—

6.3 State Management

State type	Tool	Location
Auth (user + token)	Zustand + localStorage persist	`src/contexts/authStore.ts`
Server data (queries)	TanStack Query	Inline `useQuery` / `useMutation` in page components
Local UI state	`useState`	Component level

The auth store is accessed outside React via `useAuthStore.getState()` in the Axios interceptor, avoiding circular dependency with React hooks.

6.4 API Layer

`src/services/api.ts` creates an Axios instance with:

- `baseURL` from `VITE_API_URL` (defaults to `http://localhost:4000/api`).
- Request interceptor: injects `Authorization: Bearer <token>` from the Zustand store.
- Response interceptor: on 401 (excluding `/auth/` endpoints), clears the store and redirects to `/login`.

All API calls are wrapped in typed service modules exported from `src/services/index.ts`:

Service	Exported as
Auth	`authService`
Users	`userService`
Family	`familyService`
Announcements	`announcementService`
Messages	`messageService`
Playdates	`playdateService`

6.5 Design System

Token	Value
Brand green	`#4d9463` (dark: `#3a7049`)
Accent gold	`#f0b24f` (dark: `#d4962b`)
Body font	Nunito
Heading font	Titan One
Dark mode strategy	Tailwind `class` — toggled by adding/removing `dark` class on `<html>`
Conditional classes	`cn()` helper (clsx + tailwind-merge)

Surface and background colours are CSS variables, enabling seamless dark/light mode switching without class duplication.

---

7. Authentication and Authorization

7.1 Registration and Email Verification

1. Client POSTs `{ email, password, name, city, state }` to `/api/auth/register`.
2. Server hashes the password with bcrypt (12 rounds) and stores the user with `verified = 0`.
3. A UUID token is inserted into `email_tokens` with a 24-hour expiry; a verification email is sent via Nodemailer.
4. Client GETs `/api/auth/verify-email?token=<uuid>`.
5. Server checks the token is unused and not expired, then sets `verified = 1` and marks the token `used = 1` in a transaction.

7.2 Login

1. Client POSTs `{ email, password }`.
2. Server fetches the user, compares password with bcrypt, checks `verified = 1`.
3. On success, returns a JWT signed with `JWT_SECRET`, expiry `JWT_EXPIRES_IN` (default `7d`).
4. Token payload: `{ userId: string }`.
5. Client stores token in Zustand (persisted to localStorage as `fofa-auth`).

7.3 Protected Route Middleware

`authenticate` middleware (`backend/src/middleware/auth.middleware.ts`) extracts the `Bearer` token, verifies it with `jsonwebtoken`, and attaches `userId` to `req`. Controllers read `req.userId` directly — no further authorization helper is needed for most routes.

Ownership checks (e.g., delete own announcement) are done inline in controllers with a `WHERE id = ? AND user_id = ?` SQL pattern or an explicit `403` response.

7.4 Password Reset

1. Client POSTs email to `/api/auth/forgot-password`.
2. Server always returns HTTP 200 (enumeration prevention).
3. If the user exists, a token is inserted into `password_reset_tokens` (1-hour expiry) and an email is sent.
4. Client POSTs `{ token, password }` to `/api/auth/reset-password`.
5. Token is verified (unused + not expired), password updated, token marked used — all in a transaction.

8. File Uploads and Media

8.1 Storage

Multer stores files to the local filesystem under `UPLOADS_DIR` (default `./uploads/`). Two subdirectories are auto-created:

Directory	Used for	Size limit
`uploads/thumbnails/`	User and family member photos	5 MB
`uploads/media/`	Announcement images and videos	100 MB

Files are renamed to `<uuid>.<original-ext>` to avoid collisions and path traversal risks. Uploaded filenames are stored in the database as relative paths (e.g., `/uploads/thumbnails/abc123.jpg`).

8.2 Serving

Express serves `uploads/` as static files via `express.static` with `cross-origin` resource policy (required for `<img>` tags in the SPA). Vite proxies `/uploads` to the backend during development.

8.3 Accepted Types

Upload point	Accepted extensions
Thumbnails	`.jpg`, `.jpeg`, `.png`, `.gif`, `.webp`, `.heic`, `.heif`
Announcement media	All thumbnail types plus `.mp4`, `.mov`, `.webm`, `.m4v`, `.avi`, `.mkv`

Invalid types return HTTP 400. Files over the size limit return HTTP 413.

---

9. Email Service

`backend/src/services/email.service.ts` uses Nodemailer with Gmail SMTP (App Password authentication). Two emails are sent:

Email	Trigger	Link expiry
Verification	POST `/auth/register`	24 hours
Password reset	POST `/auth/forgot-password`	1 hour

In `NODE_ENV=test`, the email functions log the URL to stdout and return without sending. This makes tests deterministic without mocking Nodemailer.

Required environment variables:

```
GMAIL_USER=your@gmail.com
GMAIL_APP_PASSWORD=your_16_char_app_password
FRONTEND_URL=http://localhost:5173   # used to build links in emails
```

---

10. Testing Strategy

10.1 Test Layers

Layer	Tool	Location	What it covers
Backend unit/integration	Jest + ts-jest	`backend/`	Controllers, middleware, auth logic
Frontend component	Vitest + React Testing Library	`frontend/src/**/*.test.tsx`	UI behavior from the user's perspective
End-to-end	Cypress 13	`cypress/e2e/`	Full user flows against live servers

10.2 Backend Tests

Run with `npm test` in the `backend/` directory. Tests use `NODE_ENV=test` which suppresses real email sends and HTTP logging.

10.3 Frontend Tests

Run with `npm test` in the `frontend/` directory (Vitest). Configuration in `vite.config.ts`:

```
test: {
  globals: true,
  environment: 'jsdom',
  setupFiles: './src/tests/setup.ts',
  coverage: { provider: 'v8', reporter: ['text', 'lcov'] },
}
```

`src/tests/setup.ts` imports `@testing-library/jest-dom` and mocks `react-hot-toast`.

Component test files are co-located with their components. Example coverage:

Component	Test file
AnnouncementCard	`AnnouncementCard.test.tsx`
CreateAnnouncementForm	`CreateAnnouncementForm.test.tsx`
CommentsSection	`CommentsSection.test.tsx`
Navbar	`Navbar.test.tsx`
DashboardPage	`DashboardPage.test.tsx`
LoginPage	`LoginPage.test.tsx`
MessagesPage	`MessagesPage.test.tsx`

10.4 End-to-End Tests

Cypress specs map one-to-one to use cases. Both servers must be running before Cypress executes.

File	Use case
`01-registration.cy.ts`	UC-01: User Registration
`02-login.cy.ts`	UC-02: Login
`03-announcements.cy.ts`	UC-03: Announcements feed
`04-family.cy.ts`	UC-04: Family management
`05-messaging.cy.ts`	UC-05: Direct messaging

File	Use case
`06-profile.cy.ts`	UC-06: Profile editing
`07-community-navigation.cy.ts`	UC-07: Community search
`08-playdates.cy.ts`	UC-08: Playdate scheduling
`09-member-profile.cy.ts`	UC-09: View member profiles

10.5 Running Tests

```
# Backend unit tests
cd backend && npm test

# Frontend unit tests with coverage
cd frontend && npm test

# E2E (both servers must be running)
cd frontend && npm run cypress:open   # interactive
cd frontend && npm run cypress:run    # headless CI
```

---

11. Deployment

11.1 Prerequisites

Requirement	Version
Node.js	18 or later
npm	8 or later
Gmail account with App Password	—
A Linux/macOS server or VM	—

SQLite requires no separate database server. No managed cloud service (RDS, S3, SES, etc.) is needed.

11.2 Environment Variables

Create `backend/.env` from `backend/.env.example`:

```
PORT=4000
JWT_SECRET=<long random string, min 32 chars>
JWT_EXPIRES_IN=7d
GMAIL_USER=your@gmail.com
GMAIL_APP_PASSWORD=<16-char app password>
FRONTEND_URL=https://your-domain.com
UPLOADS_DIR=./uploads
DB_PATH=./fofa.db
NODE_ENV=production
```

Create `frontend/.env`:

```
VITE_API_URL=https://your-domain.com/api
```

11.3 Build

```
# Backend (compile TypeScript)
cd backend
npm install --production=false
npm run build        # emits to dist/

# Frontend (Vite production build)
cd frontend
npm install
npm run build        # emits to dist/
```

The frontend build output in `frontend/dist/` is a static file tree that can be served from any web server.

11.4 Running in Production

```
# Run database migrations (idempotent — safe to re-run)
cd backend && node dist/utils/migrate.js

# Start API server
cd backend && node dist/index.js
```

The process should be managed by a process supervisor such as `pm2` or a systemd service unit to handle restarts on crash or server reboot.

Example `pm2` setup:

```
pm2 start backend/dist/index.js --name fofa-api
pm2 save
pm2 startup
```

11.5 Nginx Reverse Proxy

A typical Nginx configuration for a single server deployment:

```
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    # TLS certificates (e.g., from Let's Encrypt)
    ssl_certificate     /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # Serve React SPA
    root /var/www/fofa/frontend/dist;
    index index.html;
    try_files $uri $uri/ /index.html;

    # Proxy API calls
    location /api/ {
        proxy_pass http://localhost:4000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Proxy uploaded files
    location /uploads/ {
        proxy_pass http://localhost:4000;
```

```
    }
}
```

11.6 File Persistence

Uploaded files are written to `UPLOADS_DIR` on the local disk. In a deployment with ephemeral storage (containers, some PaaS providers), this directory must be mounted to a persistent volume. The SQLite database file (`DB_PATH`) must similarly be on persistent storage.

11.7 Database Backups

Since SQLite is a single file, backups can be taken with:

```
# Online backup (safe while server is running, uses WAL checkpoint)
sqlite3 /path/to/fofa.db ".backup /backups/fofa-$(date +%Y%m%d).db"
```

This can be run as a cron job. The backup file is a fully self-contained copy of the database.

11.8 Upgrade Procedure

1. Pull the latest code.
2. Install dependencies: `npm install` in `backend/` and `frontend/`.
3. Build: `npm run build` in both directories.
4. Run migrations: `node backend/dist/utils/migrate.js` (migrations are additive `CREATE TABLE IF NOT EXISTS` — safe to re-run against an existing database).
5. Restart the API process: `pm2 restart fofa-api`.
6. Copy the new frontend build to the web root.

---

12. Security Considerations

Area	Measure
Passwords	bcrypt with 12 rounds
JWT	Signed with `JWT_SECRET`; 7-day expiry; no refresh token in this version
Transport	HTTPS enforced at Nginx; HTTP → HTTPS redirect

Area	Measure
HTTP headers	Helmet sets `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, HSTS, etc.
Rate limiting	`express-rate-limit`: 200 requests per 15-minute window per IP
CORS	Restricted to `FRONTEND_URL`; `credentials: true`
Account enumeration	`forgotPassword` always returns HTTP 200 regardless of whether the email exists
File uploads	Extension allowlist; size limits (5 MB thumbnails, 100 MB media); UUID filenames prevent path traversal
Input validation	`express-validator` on all mutation routes; validated before controllers run
SQL injection	All queries use `better-sqlite3` parameterized statements (no string interpolation)
Token single-use	Both email verification and password reset tokens have `used` flag; consumed atomically in a transaction

13. Known Limitations and Future Work

Item	Notes
No real-time messaging	Messages require a page refresh or periodic poll to surface new content. WebSockets or SSE would be the next step.
Local file storage	Media uploaded to disk is not replicated. Moving to object storage (S3-compatible) would enable horizontal scaling and CDN delivery.
Single SQLite instance	SQLite does not support multiple concurrent writers across processes. Acceptable for a single-node deployment; PostgreSQL would be needed for horizontal scaling.
No refresh token	JWT tokens are long-lived (7 days). Adding a short-lived access token + refresh token rotation would improve security.
Mobile feature gap	The React Native app (`mobile/`) shares the API but does not implement all web features (playdates, community search).

Item	Notes
No admin interface	Moderation tools (hide posts, ban users) do not exist. Community management relies on manual database access.
Email provider coupling	Nodemailer is hardcoded to Gmail SMTP. Abstracting to a transactional email provider (SendGrid, Resend) would improve deliverability.
No image thumbnailing	Profile photos and announcement images are stored at full uploaded size. Resizing on upload would reduce storage and bandwidth costs.